Intern Assignment: Distributed Job Scheduler

Objective

Design and build a production-inspired distributed job scheduling platform capable of reliably executing asynchronous background jobs across multiple workers. The project evaluates backend engineering, database design, concurrency, reliability, API design, and full-stack implementation rather than feature count.

Core Requirements

- Implement authentication and project management. Each project can own multiple job queues.
- Support queue configuration including priority, concurrency limits, retry policy, pause/resume, and statistics.
- Allow users to create immediate, delayed, scheduled, recurring (cron), and batch jobs through REST APIs.
- Build a worker service that polls queues, atomically claims jobs, executes them concurrently, sends heartbeats, and supports graceful shutdown.
- Implement the complete job lifecycle: Queued → Scheduled → Claimed → Running → Completed, with retries and Dead Letter Queue support for permanent failures.
- Support configurable retry strategies such as fixed delay, linear backoff, and exponential backoff.
- Maintain execution logs, retry history, worker assignment, timestamps, and execution metrics for every job.
- Create a web dashboard to manage queues, inspect jobs, monitor workers, retry failed jobs, and visualize throughput and system health.

Database Design

Design an efficient relational schema for Users, Organizations, Projects, Queues, Jobs, Job Executions, Retry Policies, Workers, Worker Heartbeats, Job Logs, Scheduled Jobs, and Dead Letter Queue entries. Explain primary keys, foreign keys, indexes, normalization, cascading behavior, and performance considerations.

Backend Expectations

Expose clean REST APIs with validation, authentication, pagination, filtering, structured error handling, and logging. Ensure jobs are claimed atomically to prevent duplicate execution and make execution idempotent wherever appropriate.

Frontend Expectations

Develop a responsive dashboard showing queue health, worker status, job explorer, execution logs, queue configuration, and metrics. Live updates may be implemented using polling or WebSockets.

Bonus Features

- Workflow dependencies
- Rate limiting
- Distributed locking
- Queue sharding
- Event-driven execution
- WebSocket live updates
- Role-based access control

- AI-generated failure summaries

Deliverables

- Source code with setup instructions
- Architecture diagram
- ER diagram
- API documentation
- Design decisions document describing major trade-offs
- Automated tests for critical functionality

Evaluation Criteria	Marks
System Architecture	20
Database Design	20
Backend Engineering	20
Reliability & Concurrency	15
Frontend & UX	10
API Design	5
Documentation	5
Testing	5

Evaluation will prioritize engineering quality, modular architecture, database design, reliability, concurrency handling, observability, documentation, and maintainability over simply implementing the largest number of features.